

# Задачи на указатели и массивы

## Задача 1: Базовые операции с указателями и массивами

```
#include <iostream>
using namespace std;

void basicPointerOperations() {
    /*
    ЗАДАЧА: Понять основы работы с указателями.
    ВОПРОСЫ :
    1. Что такое указатель?
    2. Как получить адрес переменной?
    3. Как получить значение по указателю?
    4. Что такое разыменование?
    */

    cout << "=== ЗАДАЧА 1: БАЗОВЫЕ ОПЕРАЦИИ ===" << endl;
    cout << "===== " << endl;

    // 1. Создаем переменную и указатель на нее
    int number = 42;
    int* ptr = &number; // & - оператор взятия адреса

    cout << "1. Базовая работа:" << endl;
    cout << "    number = " << number << endl;
    cout << "    &number = " << &number << " (адрес number)" << endl;
    cout << "    ptr = " << ptr << " (хранит адрес number)" << endl;
    cout << "    *ptr = " << *ptr << " (разыменование - значение)" << endl;

    // 2. Изменяем значение через указатель
    *ptr = 100; // Меняем number через указатель
    cout << "\n2. После *ptr = 100:" << endl;
    cout << "    number = " << number << " (изменилось!)" << endl;
    cout << "    *ptr = " << *ptr << endl;
}
```

```

// 3. Указатель на другой тип
double price = 99.99;
double* pricePtr = &price;
cout << "\n3. Указатель на double:" << endl;
cout << "    price = " << price << endl;
cout << "    *pricePtr = " << *pricePtr << endl;

// 4. Указатель на указатель
int** ptrToPtr = &ptr;
cout << "\n4. Указатель на указатель:" << endl;
cout << "    ptrToPtr = " << ptrToPtr << " (адрес указателя)" << endl;
cout << "    *ptrToPtr = " << *ptrToPtr << " (значение ptr)" << endl;
cout << "    **ptrToPtr = " << **ptrToPtr << " (значение price)" << endl;

// 5. Null указатель
int* nullPtr = nullptr;
cout << "\n5. Null указатель:" << endl;
cout << "    nullPtr = " << nullPtr << endl;
// *nullPtr = 5; // ОШИБКА: segmentation fault!

// 6. Проверка перед использованием
if (nullPtr != nullptr) {
    *nullPtr = 10; // Не выполнится
} else {
    cout << "    Указатель равен nullptr, использовать нельзя" << endl;
}
}

void arrayAndPointerRelationship() {
    /*
    ЗАДАЧА: Понять связь массивов и указателей.
    ВОПРОСЫ:
    1. Как массив связан с указателем?
    2. Что такое арифметика указателей?
    3. Как получить доступ к элементам массива через указатели?
    */

    cout << "\n\n=== ЗАДАЧА 2: МАССИВЫ И УКАЗАТЕЛИ ===" << endl;
    cout << "===== " << endl;

    int arr[5] = {10, 20, 30, 40, 50};

```

```

// 1. Имя массива - указатель на первый элемент
cout << "1. Имя массива как указатель:" << endl;
cout << "    arr = " << arr << " (адрес первого элемента)"
cout << "    &arr[0] = " << &arr[0] << " (то же самое!)" << endl;
cout << "    *arr = " << *arr << " (первый элемент)" << endl;

// 2. Арифметика указателей
cout << "\n2. Арифметика указателей:" << endl;
int* ptr = arr; // ptr указывает на arr[0]

cout << "    ptr = " << ptr << " → *ptr = " << *ptr << endl;
ptr++; // Перемещаемся к следующему элементу
cout << "    После ptr++:" << endl;
cout << "    ptr = " << ptr << " → *ptr = " << *ptr << " (адрес элемента)" << endl;

ptr += 2; // Перемещаемся на 2 элемента вперед
cout << "    После ptr += 2:" << endl;
cout << "    ptr = " << ptr << " → *ptr = " << *ptr << " (адрес элемента)" << endl;

ptr--; // Назад на один элемент
cout << "    После ptr--:" << endl;
cout << "    ptr = " << ptr << " → *ptr = " << *ptr << " (адрес элемента)" << endl;

// 3. Разные способы доступа к элементам
cout << "\n3. Способы доступа к элементам:" << endl;
cout << "    arr[2] = " << arr[2] << endl;
cout << "    *(arr + 2) = " << *(arr + 2) << endl;
cout << "    2[arr] = " << 2[arr] << " (работает, но не рекомендуется)" << endl;

// 4. Итерация по массиву через указатели
cout << "\n4. Итерация через указатели:" << endl;
cout << "    Массив: ";
for (int* p = arr; p < arr + 5; p++) {
    cout << *p << " ";
}
cout << endl;

// 5. Указатель на весь массив vs указатель на элемент
cout << "\n5. Разница указателей:" << endl;
int (*ptrToArray)[5] = &arr; // Указатель на массив из 5 int
int* ptrToElement = arr;      // Указатель на int

cout << "    sizeof(ptrToArray) = " << sizeof(ptrToArray) << " (размер массива)" << endl;

```

```
    cout << "    sizeof(ptrToElement) = " << sizeof(ptrToElement) << endl;
    cout << "    sizeof(arr) = " << sizeof(arr) << " (размер в байтах)" << endl;
}
```

## Задача 2: Функции для работы с массивами

```
#include <iostream>
#include <algorithm> // для std::swap
using namespace std;

// 1. Функция печати массива
void printArray(int arr[], int size) {
    cout << "[";
    for (int i = 0; i < size; i++) {
        cout << arr[i];
        if (i < size - 1) cout << ", ";
    }
    cout << "]" << endl;
}

// 2. Функция поиска элемента (линейный поиск)
int findElement(int arr[], int size, int target) {
    // Алгоритм: проходим по всем элементам
    // Сложность: O(n) - линейное время
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i; // Нашли, возвращаем индекс
        }
    }
    return -1; // Не нашли
}

// 3. Функция реверса массива (на месте)
void reverseArray(int arr[], int size) {
    // Алгоритм: два указателя с двух концов
    // Сложность: O(n/2) = O(n)
    int start = 0;
    int end = size - 1;

    while (start < end) {
```

```
        // Меняем местами
        swap(arr[start], arr[end]);
        // Двигаем указатели к центру
        start++;
        end--;
    }
}

// 4. Функция нахождения максимума
int findMax(int arr[], int size) {
    if (size == 0) return -1; // Защита от пустого массива

    // Алгоритм: предполагаем первый элемент максимальным
    // Сложность: O(n)
    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

// 5. Функция нахождения минимума
int findMin(int arr[], int size) {
    if (size == 0) return -1;

    int min = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    return min;
}

// 6. Функция суммы элементов
int arraySum(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return sum;
}
```

```
}

// 7. Функция среднего значения
double arrayAverage(int arr[], int size) {
    if (size == 0) return 0.0;
    return static_cast<double>(arraySum(arr, size)) / size;
}

// 8. Функция проверки, отсортирован ли массив
bool isSorted(int arr[], int size, bool ascending = true) {
    if (size < 2) return true;

    if (ascending) {
        // Проверка сортировки по возрастанию
        for (int i = 1; i < size; i++) {
            if (arr[i] < arr[i - 1]) {
                return false;
            }
        }
    } else {
        // Проверка сортировки по убыванию
        for (int i = 1; i < size; i++) {
            if (arr[i] > arr[i - 1]) {
                return false;
            }
        }
    }
    return true;
}

// 9. Функция подсчета четных элементов
int countEven(int arr[], int size) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (arr[i] % 2 == 0) {
            count++;
        }
    }
    return count;
}

// 10. Функция копирования массива
void copyArray(int source[], int destination[], int size) {
```

```

        for (int i = 0; i < size; i++) {
            destination[i] = source[i];
        }
    }

void demonstrateArrayFunctions() {
    cout << "\n\n=== ЗАДАЧА 3: ФУНКЦИИ ДЛЯ МАССИВОВ ===" << endl;
    cout << "===== " << endl;

    int arr[] = {5, 2, 8, 1, 9, 3, 7, 4, 6};
    int size = 9;

    cout << "Исходный массив: ";
    printArray(arr, size);
    cout << endl;

    // Тестируем функции
    cout << "\n1. Поиск элемента 8:" << endl;
    int index = findElement(arr, size, 8);
    cout << "    Индекс: " << index << " (ожидается 2)" << endl;

    cout << "\n2. Поиск элемента 10 (которого нет):" << endl;
    index = findElement(arr, size, 10);
    cout << "    Индекс: " << index << " (ожидается -1)" << endl;

    cout << "\n3. Максимальный элемент:" << endl;
    cout << "    max = " << findMax(arr, size) << endl;

    cout << "\n4. Минимальный элемент:" << endl;
    cout << "    min = " << findMin(arr, size) << endl;

    cout << "\n5. Сумма элементов:" << endl;
    cout << "    sum = " << arraySum(arr, size) << endl;

    cout << "\n6. Среднее значение:" << endl;
    cout << "    average = " << arrayAverage(arr, size) << endl;

    cout << "\n7. Количество четных элементов:" << endl;
    cout << "    even count = " << countEven(arr, size) << endl;

    cout << "\n8. Проверка сортировки:" << endl;
    cout << "    Отсортирован? " << (isSorted(arr, size) ? "Да" : "Нет") << endl;
}

```

```

// Создаем отсортированный массив для теста
int sortedArr[] = {1, 2, 3, 4, 5};
cout << "    Отсортированный массив [1,2,3,4,5]: ";
cout << (isSorted(sortedArr, 5) ? "Да" : "Нет") << endl;

cout << "\n9. Копирование массива:" << endl;
int copy[9];
copyArray(arr, copy, size);
cout << "    Копия: ";
printArray(copy, size);
cout << endl;

cout << "\n10. Реверс массива:" << endl;
reverseArray(arr, size);
cout << "    После реверса: ";
printArray(arr, size);
cout << endl;

// Восстанавливаем исходный порядок
reverseArray(arr, size);

cout << "\n=== БОНУС: ПРОВЕРКА    ===" << endl;
cout << "Как работает функция swap?" << endl;
int a = 10, b = 20;
cout << "    До swap: a = " << a << ", b = " << b << endl;
swap(a, b);
cout << "    После swap: a = " << a << ", b = " << b << endl;
}

```

## Задача 3: Алгоритмы (указатели)

```

#include <iostream>
#include <cstring> // для strlen, strcpy
using namespace std;

// 1. Функция для обмена значений (работа с указателями)
void swapValues(int* a, int* b) {
    // Классический вопрос: как работает swap?
    int temp = *a; // Сохраняем значение по адресу a
    *a = *b;       // Записываем значение b в a
    *b = temp;     // Восстанавливаем старое значение a в b
}

```



```
}
```

```
// 2. Функция для обмена без временной переменной
```

```
void swapWithoutTemp(int* a, int* b) {  
    // Трюк с XOR (побитовое исключающее ИЛИ)  
  
    *a = *a ^ *b; // a теперь содержит a XOR b  
    *b = *a ^ *b; // b = (a XOR b) XOR b = a  
    *a = *a ^ *b; // a = (a XOR b) XOR a = b  
}
```

```
// 3. Функция для подсчета длины строки (аналог strlen)
```

```
int stringLength(const char* str) {  
    // str - указатель на начало строки  
    // Идем пока не встретим нулевой символ '\0'  
    int length = 0;  
    while (*str != '\0') {  
        length++;  
        str++; // Перемещаем указатель на следующий символ  
    }  
    return length;  
}
```

```
// 4. Функция для копирования строки (аналог strcpy)
```

```
void stringCopy(char* dest, const char* src) {  
    // Копируем пока не встретим '\0'  
    while (*src != '\0') {  
        *dest = *src; // Копируем символ  
        dest++;      // Двигаем указатель приемника  
        src++;        // Двигаем указатель источника  
    }  
    *dest = '\0'; // Завершаем строку нулевым символом  
}
```

```
// 5. Функция для сравнения строк (аналог strcmp)
```

```
int stringCompare(const char* str1, const char* str2) {  
    // Сравниваем посимвольно  
    while (*str1 != '\0' && *str2 != '\0') {  
        if (*str1 != *str2) {  
            return *str1 - *str2; // Разница символов  
        }  
        str1++;  
        str2++;  
    }
```

```
    }
    // Если одна строка закончилась раньше
    return *str1 - *str2;
}

// 6. Функция для конкатенации строк (аналог strcat)
void stringConcatenate(char* dest, const char* src) {
    // Находим конец первой строки
    while (*dest != '\0') {
        dest++;
    }
    // Копируем вторую строку в конец первой
    while (*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0'; // Завершаем строку
}

// 7. Функция для реверса строки на месте
void reverseString(char* str) {
    if (str == nullptr) return;

    // Находим конец строки
    char* end = str;
    while (*end != '\0') {
        end++;
    }
    end--; // Указатель на последний символ (не '\0')

    // Меняем символы с начала и конца
    char* start = str;
    while (start < end) {
        char temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }
}

// 8. Функция для проверки палиндрома
```

```

bool isPalindrome(const char* str) {
    if (str == nullptr) return false;

    // Находим конец строки
    const char* end = str;
    while (*end != '\0') {
        end++;
    }
    end--; // Последний символ

    // Сравниваем с начала и конца
    while (str < end) {
        if (*str != *end) {
            return false;
        }
        str++;
        end--;
    }
    return true;
}

// 9. Функция для нахождения подстроки (упрощенная)
const char* findSubstring(const char* str, const char* substr)
// Если подстрока пустая
if (*substr == '\0') return str;

// Проходим по основной строке
for (int i = 0; str[i] != '\0'; i++) {
    // Проверяем, совпадает ли подстрока с текущей позиции
    int j = 0;
    while (substr[j] != '\0' && str[i + j] == substr[j]) {
        j++;
    }
    // Если дошли до конца подстроки - нашли
    if (substr[j] == '\0') {
        return &str[i];
    }
}
return nullptr; // Не нашли
}

void demonstrateStringFunctions() {
    cout << "\n\n=== ЗАДАЧА 4: РАБОТА СО СТРОКАМИ ===" << endl

```

```

cout << "===== " << endl;

// 1. Работа с swap
cout << "1. Обмен значений через указатели:" << endl;
int x = 5, y = 10;
cout << "    До обмена: x = " << x << ", y = " << y << endl;
swapValues(&x, &y);
cout << "    После swapValues: x = " << x << ", y = " << y << endl;

// Возвращаем значения
swapValues(&x, &y);
cout << "    Возвращаем обратно..." << endl;

// 2. Обмен без временной переменной
cout << "\n2. Обмен без временной переменной (XOR):" << endl;
cout << "    До обмена: x = " << x << ", y = " << y << endl;
swapWithoutTemp(&x, &y);
cout << "    После swapWithoutTemp: x = " << x << ", y = " << y << endl;
swapWithoutTemp(&x, &y); // Возвращаем обратно

// 3. Длина строки
cout << "\n3. Длина строки:" << endl;
const char* text = "Hello";
cout << "    Строка: \" " << text << "\" " << endl;
cout << "    Длина (наша функция): " << strlen(text) << endl;
cout << "    Длина (strlen): " << strlen(text) << endl;

// 4. Копирование строк
cout << "\n4. Копирование строк:" << endl;
char source[] = "World";
char destination[20];
strcpy(destination, source);
cout << "    Исходная: \" " << source << "\" " << endl;
cout << "    Копия: \" " << destination << "\" " << endl;

// 5. Сравнение строк
cout << "\n5. Сравнение строк:" << endl;
cout << "    \"Hello\" vs \"Hello\": " << strcmp("Hello", "Hello") << endl;
cout << "    \"Apple\" vs \"Banana\": " << strcmp("Apple", "Banana") << endl;
cout << "    \"Zebra\" vs \"Apple\": " << strcmp("Zebra", "Apple") << endl;

// 6. Конкатенация
cout << "\n6. Конкатенация строк:" << endl;

```

```

char str1[20] = "Hello, ";
const char* str2 = "World!";
stringConcatenate(str1, str2);
cout << "    Результат: \"" << str1 << "\"" << endl;

// 7. Реверс строки
cout << "\n7. Реверс строки:" << endl;
char revStr[] = "ABCDE";
cout << "    До реверса: \"" << revStr << "\"" << endl;
reverseString(revStr);
cout << "    После реверса: \"" << revStr << "\"" << endl;

// 8. Проверка палиндрома
cout << "\n8. Проверка палиндрома:" << endl;
cout << "    \"radar\": " << (isPalindrome("radar")) ? "палиндром" << endl;
cout << "    \"hello\": " << (isPalindrome("hello")) ? "палиндром" << endl;
cout << "    \"racecar\": " << (isPalindrome("racecar")) ? "палиндром" << endl;

// 9. Поиск подстроки
cout << "\n9. Поиск подстроки:" << endl;
const char* mainStr = "Hello, beautiful world!";
const char* subStr = "beautiful";
const char* found = findSubstring(mainStr, subStr);
if (found) {
    cout << "    Нашли \"" << subStr << "\" в \"" << mainStr << endl;
    cout << "    Начинается с позиции: " << (found - mainStr) << endl;
} else {
    cout << "    Не нашли" << endl;
}
}

```

## Задача 4: Указатели на функции

```

#include <iostream>
using namespace std;

// Простые функции для демонстрации
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }

```

```

int divide(int a, int b) { return (b != 0) ? a / b : 0; }

// Функция, которая принимает указатель на функцию
int calculate(int a, int b, int (*operation)(int, int)) {
    return operation(a, b); // Вызываем функцию через указатель
}

// Еще один пример: функция для применения операции ко всем элементам массива
void applyToArray(int arr[], int size, int (*transform)(int))
    for (int i = 0; i < size; i++) {
        arr[i] = transform(arr[i]);
    }
}

// Функции-трансформации
int square(int x) { return x * x; }
int cube(int x) { return x * x * x; }
int doubleValue(int x) { return x * 2; }

void demonstrateFunctionPointers() {
    cout << "\n\n=== ЗАДАЧА 5: УКАЗАТЕЛИ НА ФУНКЦИИ ===" << endl;
    cout << "===== " << endl;

    // 1. Объявление указателя на функцию
    cout << "1. Указатель на функцию:" << endl;

    // Объявляем указатель на функцию, которая принимает два целых числа и возвращает целое
    int (*funcPtr)(int, int);

    // Присваиваем адрес функции add
    funcPtr = &add; // & не обязателен, можно просто funcPtr = add;

    // Вызываем через указатель
    int result = funcPtr(5, 3);
    cout << "    add(5, 3) через указатель: " << result << endl;

    // Меняем указатель на другую функцию
    funcPtr = subtract;
    result = funcPtr(10, 4);
    cout << "    subtract(10, 4) через указатель: " << result << endl;

    // 2. Массив указателей на функции
    cout << "\n2. Массив указателей на функции:" << endl;
}

```

```

// Массив из 4 указателей на функции
int (*operations[4])(int, int) = {add, subtract, multiply, divide};
char* operationNames[] = {"add", "subtract", "multiply", "divide"};

int a = 12, b = 4;
for (int i = 0; i < 4; i++) {
    result = operations[i](a, b);
    cout << "    " << operationNames[i] << "(" << a << ", " << b << " = " << result << endl;
}

// 3. Функция, принимающая указатель на функцию как параметр
cout << "\n3. Функция с callback:" << endl;
cout << "    calculate(8, 2, add) = " << calculate(8, 2, add) << endl;
cout << "    calculate(8, 2, multiply) = " << calculate(8, 2, multiply) << endl;

// 4. Применение функции ко всем элементам массива
cout << "\n4. Применение функции к массиву:" << endl;

int numbers[] = {1, 2, 3, 4, 5};
int size = 5;

cout << "    Исходный массив: ";
for (int i = 0; i < size; i++) cout << numbers[i] << " ";
cout << endl;

// Применяем square
int numbersCopy[5];
copy(numbers, numbers + size, numbersCopy);
applyToArray(numbersCopy, size, square);
cout << "    После square: ";
for (int i = 0; i < size; i++) cout << numbersCopy[i] << " ";
cout << endl;

// Применяем doubleValue
copy(numbers, numbers + size, numbersCopy);
applyToArray(numbersCopy, size, doubleValue);
cout << "    После doubleValue: ";
for (int i = 0; i < size; i++) cout << numbersCopy[i] << " ";
cout << endl;

// Применяем cube
copy(numbers, numbers + size, numbersCopy);

```

```

    applyToArray(numbersCopy, size, cube);
    cout << "    После cube: ";
    for (int i = 0; i < size; i++) cout << numbersCopy[i] << " ";
    cout << endl;

    // 5. typedef для указателей на функции (для удобства)
    cout << "\n5. typedef для указателей на функции:" << endl;

    // Создаем псевдоним
    typedef int (*MathOperation)(int, int);

    // Используем псевдоним
    MathOperation op1 = add;
    MathOperation op2 = multiply;

    cout << "    op1(6, 7) = " << op1(6, 7) << endl;
    cout << "    op2(6, 7) = " << op2(6, 7) << endl;
}

```

## Задача 5: Работа с динамической памятью

```

#include <iostream>
using namespace std;

void demonstrateDynamicMemory() {
    cout << "\n\n=== ЗАДАЧА 6: ДИНАМИЧЕСКАЯ ПАМЯТЬ ===" << endl;
    cout << "===== " << endl;

    // 1. Выделение памяти для одного значения
    cout << "1. Выделение памяти для одного int:" << endl;
    int* single = new int(42); // Выделяем и инициализируем :
    cout << "    Выделена память по адресу: " << single << endl;
    cout << "    Значение: " << *single << endl;

    // Меняем значение
    *single = 100;
    cout << "    После изменения: " << *single << endl;

    // 2. Выделение памяти для массива
    cout << "\n2. Выделение памяти для массива:" << endl;
}

```



```
int size = 5;
int* array = new int[size]; // Массив из 5 int

// Заполняем массив
for (int i = 0; i < size; i++) {
    array[i] = i * 10;
}

cout << "    Массив: ";
for (int i = 0; i < size; i++) {
    cout << array[i] << " ";
}
cout << endl;

// 3. Изменение размера массива (вручную)
cout << "\n3. Изменение размера массива:" << endl;

// Создаем новый массив большего размера
int newSize = 8;
int* newArray = new int[newSize];

// Копируем старые данные
for (int i = 0; i < size; i++) {
    newArray[i] = array[i];
}

// Заполняем новые элементы
for (int i = size; i < newSize; i++) {
    newArray[i] = i * 10;
}

// Освобождаем старую память
delete[] array;

// Переназначаем указатель
array = newArray;
size = newSize;

cout << "    Новый массив (размер " << size << "): ";
for (int i = 0; i < size; i++) {
    cout << array[i] << " ";
}
cout << endl;
```

```
// 4. Двумерный динамический массив
cout << "\n4. Двумерный динамический массив:" << endl;
int rows = 3, cols = 4;

// Выделяем память для указателей на строки
int** matrix = new int*[rows];

// Для каждой строки выделяем память для столбцов
for (int i = 0; i < rows; i++) {
    matrix[i] = new int[cols];

    // Заполняем значениями
    for (int j = 0; j < cols; j++) {
        matrix[i][j] = i * 10 + j;
    }
}

cout << "    Матрица " << rows << "x" << cols << ":" << endl;
for (int i = 0; i < rows; i++) {
    cout << "    ";
    for (int j = 0; j < cols; j++) {
        cout << matrix[i][j] << "\t";
    }
    cout << endl;
}

// 5. ОСВОБОЖДЕНИЕ памяти (ОЧЕНЬ ВАЖНО!)
cout << "\n5. Освобождение памяти:" << endl;

// Освобождаем single
delete single;
single = nullptr;
cout << "    Освобождена память для single" << endl;

// Освобождаем одномерный массив
delete[] array;
array = nullptr;
cout << "    Освобождена память для array" << endl;

// Освобождаем двумерный массив (в обратном порядке!)
for (int i = 0; i < rows; i++) {
    delete[] matrix[i];
}
```

```

    }
    delete[] matrix;
    matrix = nullptr;
    cout << "    Освобождена память для matrix" << endl;

    // 6. Проверка на утечки памяти
    cout << "\n6. Правила работы с динамической памятью:" << endl;
    cout << "    - Каждому new должен соответствовать delete" << endl;
    cout << "    - Каждому new[] должен соответствовать delete[]" << endl;
    cout << "    - После delete устанавливайте указатель в nullptr" << endl;
    cout << "    - Проверяйте указатель перед использованием" << endl;

    // 7. Пример опасностей
    cout << "\n7. Опасности (что НЕ делать):" << endl;

    // Утечка памяти (memory leak)
    int* leak = new int(777);
    // Забыли: delete leak;
    cout << "    Утечка: выделили память, но не освободили" << endl;

    // Двойное освобождение (double free)
    int* danger = new int(888);
    delete danger;
    // delete danger; // ОШИБКА! Уже освободили
    cout << "    Двойное освобождение: delete дважды" << endl;

    // Доступ после освобождения (use after free)
    int* zombie = new int(999);
    delete zombie;
    // *zombie = 1000; // ОШИБКА! Обращение к освобожденной памяти
    cout << "    Использование после освобождения" << endl;

    // Исправляем: освобождаем leak
    delete leak;
    leak = nullptr;
}

```

## Главная программа для тестирования

```

int main() {
    cout << "=== ТРЕНИРОВКА    ===" << endl;

```

```

cout << "Указатели и массивы в C++" << endl;
cout << "======" << endl;

int choice;

do {
    cout << "\n\nВЫБЕРИТЕ ЗАДАЧУ:" << endl;
    cout << "1. Базовые операции с указателями" << endl;
    cout << "2. Массивы и указатели" << endl;
    cout << "3. Функции для работы с массивами" << endl;
    cout << "4. Работа со строками (указатели)" << endl;
    cout << "5. Указатели на функции" << endl;
    cout << "6. Динамическая память" << endl;
    cout << "0. Выход" << endl;
    cout << "Ваш выбор: ";
    cin >> choice;

    switch(choice) {
        case 1: basicPointerOperations(); break;
        case 2: arrayAndPointerRelationship(); break;
        case 3: demonstrateArrayFunctions(); break;
        case 4: demonstrateStringFunctions(); break;
        case 5: demonstrateFunctionPointers(); break;
        case 6: demonstrateDynamicMemory(); break;
        case 0: cout << "Выход из программы" << endl; break;
        default: cout << "Неверный выбор!" << endl;
    }

    if (choice != 0) {
        cout << "\n\nНажмите Enter для продолжения...";
        cin.ignore();
        cin.get();
    }

} while (choice != 0);

cout << "1. В чем разница между указателем и ссылкой?" << endl;
cout << "2. Что такое разыменование указателя?" << endl;
cout << "3. Как работает арифметика указателей?" << endl;
cout << "4. Что такое динамическая память?" << endl;
cout << "5. Как избежать утечек памяти?" << endl;
cout << "6. Что такое nullptr и зачем он нужен?" << endl;

```

```
    cout << "7. Как передать массив в функцию?" << endl;  
    cout << "8. Что такое указатель на функцию?" << endl;  
  
    return 0;  
}
```

## Компиляция и запуск:

```
g++ -o pointers_training pointers_training.cpp  
./pointers_training
```

## Ключевые темы :

### 1. Указатели:

- Разница между \*ptr и ptr
- Операторы & и \*
- Арифметика указателей
- Null указатели

### 2. Массивы:

- Связь массивов и указателей
- Передача массивов в функции
- Разные способы доступа к элементам

### 3. Строки:

- С-строки как массивы char
- Функции для работы со строками
- Указатели при работе со строками

### 4. Динамическая память:

- new и delete
- new[] и delete[]
- Утечки памяти и как их избежать

## 5. Указатели на функции:

- Объявление и использование
- Применение в callback функциях
- Массивы указателей на функции

# Практическая работа по указателям и функциям для работы с массивами

**Цель:** Освоить основы работы с указателями и создание функций для обработки массивов в C++

## Часть 1: Основы указателей (15 минут)

### Задание 1.1: Базовые операции с указателями

```
#include <iostream> // Подключаем библиотеку для ввода/вывод

int main() { // Главная функция программы
    // Объявляем и инициализируем переменные
    int number = 42;           // Создаём целочисленную переменную
    double price = 99.99;      // Создаём переменную с плавающей точкой

    // Работа с указателями
    int* pointerToNumber = &number; // Создаём указатель на int
    double* pointerToPrice = &price; // Создаём указатель на double

    // Выводим значения и адреса
    std::cout << "=== БАЗОВЫЕ ОПЕРАЦИИ С УКАЗАТЕЛЯМИ ===" << std::endl;
    std::cout << std::endl;
```

```

// 1. Выводим значения переменных
std::cout << "1. Значения переменных:" << std::endl;
std::cout << "    number = " << number << std::endl;
std::cout << "    price = " << price << std::endl;
std::cout << std::endl;

// 2. Выводим адреса переменных
std::cout << "2. Адреса переменных в памяти:" << std::endl;
std::cout << "    &number = " << &number << std::endl;
std::cout << "    &price = " << &price << std::endl;
std::cout << std::endl;

// 3. Выводим значения указателей
std::cout << "3. Значения указателей:" << std::endl;
std::cout << "    pointerToNumber = " << pointerToNumber << std::endl;
std::cout << "    pointerToPrice = " << pointerToPrice << std::endl;
std::cout << std::endl;

// 4. Разыменование указателей
std::cout << "4. Разыменование указателей:" << std::endl;
std::cout << "    *pointerToNumber = " << *pointerToNumber << std::endl;
std::cout << "    *pointerToPrice = " << *pointerToPrice << std::endl;
std::cout << std::endl;

// 5. Изменение значения через указатель
std::cout << "5. Изменение значения через указатель:" << std::endl;
*pointerToNumber = 100;           // Меняем значение number
std::cout << "    После *pointerToNumber = 100:" << std::endl;
std::cout << "    number = " << number << std::endl;
std::cout << "    *pointerToNumber = " << *pointerToNumber << std::endl;
std::cout << std::endl;

// 6. Работа с нулевым указателем
std::cout << "6. Работа с нулевым указателем:" << std::endl;
int* nullPointer = nullptr;       // Создаём нулевой указатель
// int* nullPointer = NULL;       // Старый стиль (лучше не использовать)
// int* nullPointer = 0;          // Ещё один старый стиль

if (nullPointer == nullptr) {     // Проверяем, является ли указатель нулевым
    std::cout << "    nullPointer равен nullptr" << std::endl;
}

```

```
// НИКОГДА не разыменовывайте нулевой указатель!
// *nullPointer = 10; // ОШИБКА: программа упадёт!

return 0; // Завершаем программу успешно
}
```

## Задание 1.2: Указатели и массивы

```
#include <iostream>

int main() {
    std::cout << "=== УКАЗАТЕЛИ И МАССИВЫ ===" << std::endl;
    std::cout << std::endl;

    // Создаём массив из 5 элементов
    int numbers[5] = {10, 20, 30, 40, 50}; // Статический массив

    // 1. Имя массива - это указатель на первый элемент
    std::cout << "1. Имя массива как указатель:" << std::endl;
    std::cout << "    numbers = " << numbers << std::endl;
    std::cout << "    &numbers[0] = " << &numbers[0] << std::endl;
    std::cout << "    *numbers = " << *numbers << std::endl;
    std::cout << std::endl;

    // 2. Создаём указатель на массив
    int* ptr = numbers; // ptr указывает на numbers[0]

    std::cout << "2. Работа через указатель:" << std::endl;
    std::cout << "    ptr = " << ptr << std::endl;
    std::cout << "    *ptr = " << *ptr << std::endl; // n
    std::cout << std::endl;

    // 3. Арифметика указателей
    std::cout << "3. Арифметика указателей:" << std::endl;
    std::cout << "    ptr + 1 = " << ptr + 1 << std::endl;
    std::cout << "    *(ptr + 1) = " << *(ptr + 1) << std::endl;
    std::cout << std::endl;

    // 4. Итерация по массиву через указатель
    std::cout << "4. Итерация по массиву через указатель:" <<
    std::cout << "    Элементы массива: ";
```



```

// Первый способ: через указатель
int* current = numbers; // Начинаем с первого элемента
for (int i = 0; i < 5; i++) {
    std::cout << *current << " "; // Выводим значение
    current++;                     // Переходим к следующему
}
std::cout << std::endl;

// Второй способ: через индексы
std::cout << "    Через индексы: ";
for (int i = 0; i < 5; i++) {
    std::cout << *(numbers + i) << " "; // numbers[i] эквивалентно *(numbers + i)
}
std::cout << std::endl;

// 5. Изменение элементов через указатель
std::cout << std::endl;
std::cout << "5. Изменение элементов через указатель:" << std::endl;

ptr = numbers; // Возвращаем указатель на начало массива
*(ptr + 2) = 300; // Меняем третий элемент (индекс 2) на 300

std::cout << "    После *(ptr + 2) = 300:" << std::endl;
std::cout << "    numbers[2] = " << numbers[2] << std::endl;

return 0;
}

```

## Часть 2: Функции для работы с массивами (15 минут)

### Задание 2.1: Базовые функции обработки массивов

```

#include <iostream>

// ФУНКЦИЯ 1: Вывод массива
// Принимает указатель на массив и его размер

```

```
void printArray(int* arr, int size) {
    std::cout << "Массив [" << size << "]: ";

    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " "; // arr[i] эквивалентно *
    }

    std::cout << std::endl;
}

// ФУНКЦИЯ 2: Заполнение массива
// Принимает указатель на массив, его размер и значение для заполнения
void fillArray(int* arr, int size, int value) {
    for (int i = 0; i < size; i++) {
        arr[i] = value; // Заполняем каждый элемент заданным значением
    }
}

// ФУНКЦИЯ 3: Поиск максимального элемента
// Принимает указатель на массив и его размер
// Возвращает максимальный элемент
int findMax(int* arr, int size) {
    if (size <= 0) { // Проверка на пустой массив
        return 0; // Возвращаем 0 в случае ошибки
    }

    int max = arr[0]; // Предполагаем, что первый элемент - максимальный

    for (int i = 1; i < size; i++) { // Начинаем со второго элемента
        if (arr[i] > max) { // Если текущий элемент больше max
            max = arr[i]; // Обновляем max
        }
    }

    return max; // Возвращаем найденный максимум
}

// ФУНКЦИЯ 4: Сумма элементов массива
// Принимает указатель на массив и его размер
// Возвращает сумму всех элементов
int sumArray(int* arr, int size) {
    int sum = 0; // Инициализируем сумму нулём
```

```
        for (int i = 0; i < size; i++) {
            sum += arr[i]; // Добавляем каждый элемент к сумме
        }

        return sum; // Возвращаем сумму
    }

// ФУНКЦИЯ 5: Обратный порядок массива
// Принимает указатель на массив и его размер
// Меняет порядок элементов на обратный
void reverseArray(int* arr, int size) {
    int start = 0;           // Индекс начала
    int end = size - 1;      // Индекс конца

    while (start < end) {    // Пока не встретились в середине
        // Меняем местами элементы
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;

        // Двигаемся к центру
        start++;
        end--;
    }
}

int main() {
    std::cout << "=== ФУНКЦИИ ДЛЯ РАБОТЫ С МАССИВАМИ ===" << ;
    std::cout << std::endl;

    const int SIZE = 6; // Константа для размера массива
    int arr[SIZE];      // Объявляем массив

    // 1. Заполняем и выводим массив
    std::cout << "1. Заполнение массива:" << std::endl;
    fillArray(arr, SIZE, 0); // Заполняем нулями
    printArray(arr, SIZE);    // Выводим

    // 2. Меняем значения и снова выводим
    std::cout << std::endl;
    std::cout << "2. Изменение значений:" << std::endl;

    for (int i = 0; i < SIZE; i++) {
```

```

        arr[i] = (i + 1) * 10; // Заполняем значениями 10, 20, 30, 40, 50
    }

    printArray(arr, SIZE);

    // 3. Находим максимальный элемент
    std::cout << std::endl;
    std::cout << "3. Поиск максимального элемента:" << std::endl;

    int max = findMax(arr, SIZE);
    std::cout << "    Максимальный элемент: " << max << std::endl;

    // 4. Вычисляем сумму элементов
    std::cout << std::endl;
    std::cout << "4. Сумма элементов:" << std::endl;

    int sum = sumArray(arr, SIZE);
    std::cout << "    Сумма всех элементов: " << sum << std::endl;

    // 5. Обратный порядок
    std::cout << std::endl;
    std::cout << "5. Обратный порядок:" << std::endl;

    std::cout << "    До reverse: ";
    printArray(arr, SIZE);

    reverseArray(arr, SIZE);

    std::cout << "    После reverse: ";
    printArray(arr, SIZE);

    // 6. Среднее арифметическое (используем ранее созданные функции)
    std::cout << std::endl;
    std::cout << "6. Среднее арифметическое:" << std::endl;

    double average = static_cast<double>(sum) / SIZE; // Преобразуем в double
    std::cout << "    Среднее значение: " << average << std::endl;

    return 0;
}

```

## Задание 2.2: Функции с возвратом указателей

```
#include <iostream>

// ФУНКЦИЯ 1: Поиск элемента в массиве
// Возвращает указатель на найденный элемент или nullptr
int* findElement(int* arr, int size, int value) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == value) {           // Если нашли нужное значение
            return &arr[i];             // Возвращаем указатель на элемент
            // Можно также вернуть: return arr + i;
        }
    }

    return nullptr; // Если не нашли, возвращаем nullptr
}

// ФУНКЦИЯ 2: Создание и заполнение массива
// Возвращает указатель на динамически созданный массив
int* createArray(int size, int initialValue) {
    int* newArray = new int[size]; // Выделяем память в куче

    for (int i = 0; i < size; i++) {
        newArray[i] = initialValue; // Заполняем массив
    }

    return newArray; // Возвращаем указатель на созданный массив
}

// ФУНКЦИЯ 3: Копирование массива
// Возвращает указатель на копию массива
int* copyArray(const int* source, int size) {
    int* copy = new int[size]; // Выделяем память для копии

    for (int i = 0; i < size; i++) {
        copy[i] = source[i]; // Копируем каждый элемент
    }

    return copy; // Возвращаем указатель на копию
}

int main() {
```

```

std::cout << "=== ФУНКЦИИ С ВОЗВРАТОМ УКАЗАТЕЛЕЙ ===" << ;
std::cout << std::endl;

int staticArray[5] = {10, 20, 30, 40, 50};

// 1. Поиск элемента
std::cout << "1. Поиск элемента в массиве:" << std::endl;

int* found = findElement(staticArray, 5, 30);

if (found != nullptr) {
    std::cout << "    Найден элемент: " << *found << std::endl;
    std::cout << "    По адресу: " << found << std::endl;

    // Можем изменить найденный элемент
    *found = 300;
    std::cout << "    После изменения: staticArray[2] = " << staticArray[2] << std::endl;
} else {
    std::cout << "    Элемент не найден" << std::endl;
}

// 2. Создание динамического массива
std::cout << std::endl;
std::cout << "2. Создание динамического массива:" << std::endl;

int* dynamicArray = createArray(5, 7); // Создаём массив

std::cout << "    Динамический массив: ";
for (int i = 0; i < 5; i++) {
    std::cout << dynamicArray[i] << " ";
}
std::cout << std::endl;

// 3. Копирование массива
std::cout << std::endl;
std::cout << "3. Копирование массива:" << std::endl;

int* copiedArray = copyArray(staticArray, 5);

std::cout << "    Оригинал: ";
for (int i = 0; i < 5; i++) {
    std::cout << staticArray[i] << " ";
}

```

```

std::cout << std::endl;

std::cout << "    Копия: ";
for (int i = 0; i < 5; i++) {
    std::cout << copiedArray[i] << " ";
}
std::cout << std::endl;

// Изменяем копию (оригинал не должен измениться)
copiedArray[0] = 999;

std::cout << "    После изменения копии:" << std::endl;
std::cout << "    Оригинал[0] = " << staticArray[0] << std::endl;
std::cout << "    Копия[0] = " << copiedArray[0] << std::endl;

// ВАЖНО: освобождаем динамически выделенную память
delete[] dynamicArray; // Освобождаем первый массив
delete[] copiedArray;  // Освобождаем второй массив

// Обнуляем указатели (хорошая практика)
dynamicArray = nullptr;
copiedArray = nullptr;

std::cout << std::endl;
std::cout << "Память освобождена, указатели обнулены" << std::endl;

return 0;
}

```

### Задача 3.2: Поиск и замена в массиве

```

#include <iostream>

// Функция для поиска всех вхождений значения
// Возвращает количество найденных элементов
int findAll(const int* arr, int size, int value, int* positions) {
    int count = 0; // Счётчик найденных элементов

    for (int i = 0; i < size; i++) {
        if (arr[i] == value) { // Если нашли нужное значение
            if (positions != nullptr) { // Если передан массив для хранения позиций
                positions[count] = i;
            }
            count++;
        }
    }

    return count;
}

```

```
        positions[count] = i; // Сохраняем позицию
    }
    count++;                  // Увеличиваем счётчик
}

return count; // Возвращаем количество найденных
}

// Функция для замены всех вхождений одного значения на другое
// Возвращает количество заменённых элементов
int replaceAll(int* arr, int size, int oldValue, int newValue)
{
    int replaced = 0; // Счётчик замен

    for (int i = 0; i < size; i++) {
        if (arr[i] == oldValue) { // Если нашли старое значение
            arr[i] = newValue;    // Заменяем на новое
            replaced++;           // Увеличиваем счётчик
        }
    }

    return replaced; // Возвращаем количество замен
}

// Функция для вывода массива с подсветкой позиций
void printArrayWithHighlights(const int* arr, int size,
                             const int* positions, int posCount)
{
    std::cout << "Массив: ";

    for (int i = 0; i < size; i++) {
        bool isHighlighted = false;

        // Проверяем, нужно ли подсветить этот элемент
        for (int j = 0; j < posCount; j++) {
            if (positions[j] == i) {
                isHighlighted = true;
                break;
            }
        }

        if (isHighlighted) {
            std::cout << "[" << arr[i] << "]" "; // Подсветка
        } else {
```



```

        std::cout << arr[i] << " ";           // Обычный вы
    }
}

std::cout << std::endl;
}

int main() {
    std::cout << "=== ПОИСК И ЗАМЕНА В МАССИВЕ ===" << std::endl;
    std::cout << std::endl;

    // Создаём тестовый массив
    const int SIZE = 10;
    int array[SIZE] = {2, 5, 2, 8, 2, 9, 5, 2, 1, 2};

    // Выводим исходный массив
    std::cout << "Исходный массив: ";
    for (int i = 0; i < SIZE; i++) {
        std::cout << array[i] << " ";
    }
    std::cout << std::endl << std::endl;

    // 1. Поиск всех вхождений числа 2
    std::cout << "1. Поиск всех вхождений числа 2:" << std::endl;

    int positions[SIZE]; // Массив для хранения позиций
    int count = findAll(array, SIZE, 2, positions);

    std::cout << "    Найдено " << count << " вхождений" << std::endl;

    if (count > 0) {
        std::cout << "    Позиции: ";
        for (int i = 0; i < count; i++) {
            std::cout << positions[i] << " ";
        }
        std::cout << std::endl;

        // Выводим массив с подсветкой найденных элементов
        printArrayWithHighlights(array, SIZE, positions, count);
    }

    std::cout << std::endl;
}

```

```

// 2. Замена всех двоек на семёрки
std::cout << "2. Замена всех 2 на 7:" << std::endl;

int replaced = replaceAll(array, SIZE, 2, 7);

std::cout << "    Заменено " << replaced << " элементов" << std::endl;
std::cout << "    Массив после замены: ";

for (int i = 0; i < SIZE; i++) {
    std::cout << array[i] << " ";
}
std::cout << std::endl << std::endl;

// 3. Поиск всех пятёрок (после замены)
std::cout << "3. Поиск всех вхождений числа 5:" << std::endl;

count = findAll(array, SIZE, 5, positions);

std::cout << "    Найдено " << count << " вхождений" << std::endl;

if (count > 0) {
    printArrayWithHighlights(array, SIZE, positions, count);
}

return 0;
}

```

### Задача 3.3: Объединение двух массивов

```

#include <iostream>

// Функция для объединения двух массивов
// Возвращает указатель на новый массив
int* mergeArrays(const int* arr1, int size1, const int* arr2,
    // Вычисляем общий размер
    int totalSize = size1 + size2;

    // Выделяем память для нового массива
    int* merged = new int[totalSize];

    // Копируем элементы из первого массива

```

```
    for (int i = 0; i < size1; i++) {
        merged[i] = arr1[i];
    }

    // Копируем элементы из второго массива
    for (int i = 0; i < size2; i++) {
        merged[size1 + i] = arr2[i];
    }

    return merged; // Возвращаем указатель на новый массив
}

// Функция для сортировки массива (простой алгоритм)
void sortArray(int* arr, int size) {
    // Используем простой алгоритм сортировки пузырьком
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Меняем местами
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    std::cout << "=== ОБЪЕДИНЕНИЕ ДВУХ МАССИВОВ ===" << std::endl;
    std::cout << std::endl;

    // Создаём два массива
    int array1[] = {3, 7, 1, 9};
    int array2[] = {4, 2, 8, 6, 5};

    int size1 = sizeof(array1) / sizeof(array1[0]);
    int size2 = sizeof(array2) / sizeof(array2[0]);

    // Выводим исходные массивы
    std::cout << "Первый массив: ";
    for (int i = 0; i < size1; i++) {
        std::cout << array1[i] << " ";
    }
}
```

```
std::cout << std::endl;

std::cout << "Второй массив: ";
for (int i = 0; i < size2; i++) {
    std::cout << array2[i] << " ";
}
std::cout << std::endl << std::endl;

// Объединяем массивы
int* merged = mergeArrays(array1, size1, array2, size2);
int totalSize = size1 + size2;

std::cout << "Объединённый массив: ";
for (int i = 0; i < totalSize; i++) {
    std::cout << merged[i] << " ";
}
std::cout << std::endl << std::endl;

// Сортируем объединённый массив
sortArray(merged, totalSize);

std::cout << "Отсортированный объединённый массив: ";
for (int i = 0; i < totalSize; i++) {
    std::cout << merged[i] << " ";
}
std::cout << std::endl;

// Освобождаем память
delete[] merged;
merged = nullptr;

std::cout << std::endl;
std::cout << "Память освобождена" << std::endl;

return 0;
}
```

## Заключение и домашнее задание

1. **Основы указателей:** создание, инициализация, разыменование
2. **Связь указателей и массивов:** имя массива как указатель

3. **Функции для работы с массивами:** передача массивов в функции
4. **Динамическое выделение памяти:** операторы `new` и `delete`
5. **Практические задачи:** статистика, поиск, замена, объединение

### Домашнее задание:

1. **Задача 1:** Напишите функцию, которая находит второй по величине элемент в массиве
2. **Задача 2:** Создайте функцию, которая удаляет все повторяющиеся элементы из массива
3. **Задача 3:** Напишите программу, которая считывает массив с клавиатуры и сортирует его
4. **Задача 4:** Создайте функцию, которая проверяет, является ли массив палиндромом
5. **Задача 5:** Реализуйте простую базу данных студентов с использованием структур и массивов

### Советы для дальнейшего изучения:

1. Изучите `std::vector` для работы с динамическими массивами
2. Познакомьтесь с умными указателями ( `std::unique_ptr` , `std::shared_ptr` )
3. Изучите алгоритмы сортировки (быстрая сортировка, сортировка слиянием)
4. Практикуйтесь на сайтах типа LeetCode или CodeWars

**Важно:** Всегда проверяйте указатели на `nullptr` перед использованием и не забывайте освобождать динамически выделенную память!